

Predictive Lead Scoring for B2B Hardware + Services (Logitech-for-Business Archetype): A Build-Level Reference Architecture on Salesforce + HubSpot + Snowflake

TL;DR

- Build a **point-in-time, gradient-boosted propensity model** (LightGBM/XGBoost/CatBoost) on Snowflake that predicts **MQL→SQL or opportunity-creation within a fixed 60-90 day window**, not raw "closed-won," because long/variable B2B cycles make closed-won leaky and slow to label; layer an **expected-value score** = $P(\text{convert}) \times E[\text{deal value}]$ and reserve survival analysis for censored, long-cycle enterprise/education deals.
 - The single largest failure mode is **data leakage from querying current CRM state**; the entire design is organized around an as-of-date feature store (Snowflake `ASOF JOIN`, streams/snapshots, dynamic tables), temporal cross-validation, and probability calibration, with GBDTs justified by peer-reviewed tabular-ML evidence (Grinsztajn et al., NeurIPS 2022; Shwartz-Ziv & Armon, 2022).
 - Keep training and scoring in Snowflake (Snowpark ML, Model Registry, Feature Store, Cortex `CLASSIFICATION` as a fast baseline), then **reverse-ETL scores back to Salesforce and HubSpot custom fields** (Hightouch/Census or HubSpot's native Snowflake data share), and protect against the **self-fulfilling feedback loop** with a permanent random exploration holdout.
-

1. Business Framing

1.1 What lead scoring actually predicts

A lead score is a model-estimated **probability that a lead/account will reach a defined business outcome inside a defined time window**. Three conceptually distinct scores are usually conflated:

- **Fit score** (a.k.a. firmographic/ICP fit): how well the prospect matches your ideal-customer profile — company size, industry, geography, persona seniority. This is largely static and answers "*should we sell to them?*"
- **Intent/engagement score**: how actively the prospect is behaving like a buyer right now — web visits, content downloads, demo requests, third-party intent surges. This is dynamic and answers "*are they in-market now?*"
- **Blended propensity score**: a single ML-estimated probability that fuses fit + intent + behavior + relationship history into one number.

Recommendation and reasoning: Model fit and intent as **separate feature groups feeding one blended propensity model**, but *also* expose the decomposition to the business (via SHAP groupings) so sales sees both "good fit / low intent" and "poor fit / high intent." A single blended score is superior for *ranking and routing* (one comparable number, calibrated to a real probability). Two separate scores are superior for *play selection* (a high-fit/low-intent account gets nurture; a high-intent/low-fit lead gets a fast human qualification to weed out tire-kickers). Best-in-class setups do both: one model, two lenses.

1.2 Choosing the target/label precisely

The label choice determines everything downstream. Candidate targets, in increasing distance from the model's control:

Target	Pro	Con	When to use
MQL→SQL conversion	Fast to observe (days-weeks), high positive volume, directly actionable by marketing/SDRs	Proxy for revenue, not revenue itself; depends on SDR behavior	Primary target for high-volume, top-of-funnel scoring
Opportunity creation	Still reasonably fast; strong revenue correlation; less SDR-subjectivity than SQL	Sales-created, so partly reflects rep discretion	Strong default; recommended primary label
Closed-won	The real business outcome	Long, variable latency (quarters to >1 year for enterprise/education/tender deals); sparse positives; heavily censored	Secondary/expected-value layer, or survival model
Expected value ($P \times \$$)	Aligns to revenue prioritization	Requires a value model; compounds two sources of error	Ranking layer on top of a probability model

Why "closed-won" alone is often the wrong target. For a hardware+services B2B seller with channel deals, closed-won can lag lead creation by 6-18 months (enterprise VC-room rollouts, education procurement tenders, distributor sell-through). Training a fixed-window binary classifier on "closed-won within N days" for a short N starves you of positives (almost nobody closes that fast) and biases toward atypically fast SMB deals; for a long N you cannot score recent leads at all (their outcome is unknown), and you must throw away the most recent, most relevant data. The pragmatic answer used by most mature RevOps teams: **train on opportunity-creation (or MQL→SQL) as the primary label** because it is observable quickly and correlates strongly with downstream revenue, then **handle revenue separately** via an expected-value multiplier and/or a survival model for the closed-won tail.

Handling long/variable time-to-conversion: (a) fix an outcome window aligned to a business decision cadence; (b) for the long tail, use **survival/time-to-event models** (Section 4.3) that correctly treat not-yet-converted leads as *right-censored* rather than negative; (c) segment models by motion (SMB self-serve vs. enterprise tender vs. channel) because their time-to-event distributions differ by an order of magnitude.

1.3 Prediction window and label leakage interaction

Define the label as: *given everything known as of observation date (t_0) , does the lead reach the outcome within $[t_0, t_0 + W]$* ? The window (W) must be chosen jointly with three things:

- **Sales cycle length:** (W) should cover the bulk (e.g., median to 75th percentile) of the *lead-to-*

outcome latency for that motion. If the median MQL→opportunity latency is ~34 days, a (W) of 60–90 days captures most true positives without excessive censoring.

- **Label maturity / censoring:** any lead created after $(today - W)$ has an *incomplete* window and must be **excluded from training** (its label is not yet determined) — a subtle but critical leakage guard. This shrinks usable training data and is why very long windows are costly.
- **Actionability:** (W) must match how fast the score is useful. A 90-day propensity is useless for a "call within 24 hours" SLA; that needs a short-window or event-triggered model.

1.4 Level of scoring: lead vs. contact vs. account vs. buying group

- **Lead-level** (Salesforce $(Lead)$, HubSpot $(contact)$ pre-lifecycle): the classic MQL model. Simple but ignores that B2B buys are committee decisions.
- **Contact-level:** score each person; needed for persona-based routing.
- **Account-level:** roll contacts up to the buying organization. Essential for **ABM** and for hardware deals where the *account's* refresh cycle and install base matter more than any one contact.
- **Buying-group / opportunity-level:** score the *committee* around a specific deal — the state of the art for enterprise. A single account can have several concurrent buying groups (e.g., IT refresh vs. new-site build-out).

Recommendation: Build a **two-tier scoring system**: (1) an account-fit/propensity model as the ABM backbone, and (2) a lead/contact engagement-propensity model whose scores **roll up to the account** via aggregation features (max, mean, count of engaged contacts, buying-group coverage). This mirrors how mature B2B orgs operate and is the only way to represent the buying committee.

Handling multiple contacts within one account (the committee). Engineer **account-level rollups** of contact features (e.g., "number of distinct engaged personas," "seniority of most-engaged contact," "is an economic buyer engaged?"). This is where **lead-to-account matching** becomes make-or-break: leads must be resolved to the right $(Account)$ (Salesforce $(Lead) \rightarrow (Account)$ matching, HubSpot contact→company association). Fuzzy/domain-based matching errors silently corrupt account rollups and are a top-five practical failure mode.

1.5 Relationship to a B2B CLV model

Lead scoring and CLV answer different questions and belong at different funnel stages:

- **Lead scoring:** $P(\text{this prospect converts soon})$ — an *acquisition/prioritization* model, short horizon, features dominated by fit + intent + engagement.
- **CLV:** expected long-run margin from a *won customer* — a *post-acquisition value* model, long horizon, features dominated by install base, product mix, expansion/renewal behavior.

They connect through the **expected-value score**: $(Priority = P(\text{convert within } W) \times E[\text{first-deal value}] \times (\text{optionally}) E[\text{CLV multiple}])$. Reuse the same Snowflake feature store, entity keys (account/lead IDs), point-in-time discipline, and MLOps for both; the value/CLV model becomes the $(E[\text{value}])$ term. Keep them as **separate models** — fusing them into one target destroys interpretability and mixes two very different error structures.

2. Data Sources and Field Mapping

All Salesforce API names below are confirmed against the Salesforce Object Reference; all HubSpot property names against HubSpot's default-property documentation. Where a signal must be a custom field, it is flagged explicitly.

2.1 Salesforce (system of record for sales)

Key objects and **confirmed** standard fields:

- **Lead:** `Id`, `IsConverted` (Boolean), `ConvertedOpportunityId`, `ConvertedContactId`, `ConvertedAccountId`, `ConvertedDate`, `Status`, `LeadSource`, `Rating`, `Industry`, `NumberOfEmployees`, `AnnualRevenue`, `Company`, `Title`, `Country`, `CreatedDate`, `LastActivityDate`. *Conversion fields are populated at conversion time — prime leakage sources (Section 3.4).*
- **Contact / Account:** Account has `NumberOfEmployees`, `AnnualRevenue`, `Industry`, `Sic` (SIC code; `SicDesc` for the description), `Type`, `OwnerId`.
- **Opportunity:** `Amount`, `StageName` (label "Stage"), `IsWon` (read-only), `IsClosed` (read-only), `CloseDate`, `CreatedDate`, `Type`, `LeadSource`, `AccountId`. `IsWon`/`IsClosed` are derived from the stage's mapping.
- **OpportunityLineItem:** `OpportunityId`, `PricebookEntryId`, `Product2Id`, `Quantity`, `UnitPrice` (label "Sales Price"), `TotalPrice`, `ListPrice` (read-only), `ServiceDate` (label "Date"). This is where product mix / hardware SKUs live.
- **Product2 / PricebookEntry:** product catalog and pricing; join for product-family features.
- **Campaign / CampaignMember:** CampaignMember confirmed fields — `CampaignId`, `LeadId`, `ContactId`, `Status`, `HasResponded` (Boolean, auto-set when Status maps to a "Responded" value), `FirstRespondedDate` (auto-populated when `HasResponded` becomes true), `CreatedDate` (when the member was added).
- **Task / Event (Activities):** polymorphic `WhoId` (Lead/Contact) and `WhatId` (Account/Opportunity/Campaign/Case/custom). Task fields include `ActivityDate`, `Status`, `OwnerId`; Event has start/end datetime. Source of "sales touches, calls, meetings logged, SLA-to-first-touch" features.
- **LeadHistory / OpportunityHistory / OpportunityFieldHistory:**
 - `OpportunityHistory` = **stage history**. Per Salesforce Help, a new row is added automatically whenever the **Amount, Probability, Stage, or CloseDate** fields change, admins cannot select which fields are tracked, and — unlike other field history — **opportunity stage history data isn't automatically deleted**. This is the canonical source for point-in-time stage reconstruction.
 - `OpportunityFieldHistory` = generic field-change history for the Opportunity's other tracked fields (must enable tracking).
 - `LeadHistory` = field-change history for Lead ("History for tracked fields of Lead"): fields `LeadId`, `Field` (the changed field; a special `created` entry on creation), `OldValue`, `NewValue`, `CreatedById`, `CreatedDate`.

- **Field history tracking must be explicitly enabled.** Per Salesforce Help, when Field Audit Trail is off, Salesforce retains field history for up to 18 months in the UI and up to 24 months via the API; the standard limit is 20 tracked fields per object, extendable to 60 with the Field Audit Trail add-on (Tasks and Events are limited to 6 tracked fields each). These history objects are the backbone of point-in-time feature reconstruction — enable tracking on `Lead.Status`, `Lead.Rating`, `Lead.LeadSource` now, because you cannot retroactively recover history you never tracked.
- **User:** owner/rep attributes for SLA and workload features.

2.2 HubSpot (marketing automation)

Confirmed default **contact** properties:

- Lifecycle/qualification: `lifecyclestage`, `hs_lead_status`, `hubspotscore` (rule-based score set in HubSpot's scoring app), `hs_predictivecontactscore` and `hs_predictivecontactscorebucket` (HubSpot's own predictive score/bucket — use as a *feature/benchmark*, not ground truth), plus lifecycle-date stamps `hs_lifecyclestage_lead_date`, `hs_lifecyclestage_marketingqualifiedlead_date`, `hs_lifecyclestage_salesqualifiedlead_date`, `hs_lifecyclestage_opportunity_date`, `hs_lifecyclestage_customer_date` (invaluable for point-in-time labels).
- Source/attribution: `hs_analytics_source`, `hs_analytics_source_data_1/2`, `hs_latest_source`.
- Web analytics: `hs_analytics_num_page_views`, `hs_analytics_num_visits`, `hs_analytics_num_event_completions`, `hs_analytics_first_url`, `hs_analytics_last_url`, `hs_analytics_first_timestamp`, time-last-seen.
- Email: `hs_email_open`, `hs_email_click`, `hs_email_bounce`, `hs_email_optout` (**note the Apple MPP caveat — opens are unreliable, Section 8**).
- Conversions/forms: `num_conversion_events`, `num_unique_conversion_events`, `recent_conversion_event_name`, `recent_conversion_date`, `first_conversion_event_name`.
- Deal-derived: `recent_deal_amount`, `recent_deal_close_date`, `total_revenue`, `num_associated_deals`.
- **Companies** and **deals** objects mirror these at company/deal level; **engagements/activities** (emails, calls, meetings, notes) and **email events** (sent/open/click/bounce/unsubscribe) are separate event streams. In Snowflake these arrive as semi-structured JSON (handle with `LATERAL FLATTEN` / `VARIANT`, Section 6).

2.3 Third-party / enrichment data

- **Firmographics** (ZoomInfo/Clearbit/D&B): employee count, revenue, industry (SIC/NAICS), HQ, corporate hierarchy. Adds fit signal and *fills CRM gaps* (CRM firmographics are notoriously sparse/stale). Join on domain + normalized company name.
- **Technographics** (HG Insights, BuiltWith): installed technologies (e.g., existing UC platform — Zoom/Teams/Google Meet — is a strong fit signal for VC hardware). Join on domain.
- **Third-party intent** (Bombora Company Surge, G2 buyer intent, 6sense, Demandbase): topic-level surge signals at the *account* level. Join on domain; treat as account features rolled

to leads. These capture demand happening *off* your properties.

- **First-party product/web telemetry:** your own site + product usage.

2.4 Hardware/peripherals-specific signals (mostly custom objects/fields)

These do not exist as Salesforce/HubSpot standard fields and **must be modeled as custom objects/fields** — flag them as __c:

- **Install base / prior device purchases:** a custom Install_Base__c object (or derived from OpportunityLineItem + ERP), keyed to Account.
- **Refresh/replacement cycle:** derived feature = time since last hardware purchase vs. typical device lifecycle.
- **VC room deployments:** count/type of deployed conference-room systems (custom object).
- **Partner/reseller-sourced deals & distributor sell-through:** custom fields on Opportunity (e.g., Reseller_Account__c, Distributor__c, Sell_Through_Flag__c) — critical because in channel deals the end customer is often obscured (Section 8).
- **Warranty/support tickets:** from a service system (Salesforce Case or Zendesk) — negative or positive signal depending on context.
- **Trials/demos, quote requests, RFPs:** custom objects/campaign types; strong intent signals.
- **Education/enterprise procurement cycle:** fiscal-year/tender seasonality features.

2.5 Field-mapping table (source → object → field/API name → derived feature → rationale)

Source	Object	Field / API name	Derived feature	Rationale
Salesforce	Account	<code>NumberOfEmployees</code> , <code>AnnualRevenue</code>	log-scaled size bands	Firmographic fit; nonlinear so bin/log
Salesforce	Account	<code>Industry</code> , <code>Sic</code>	target-encoded industry (in-fold)	Vertical strongly predicts fit for hardware
Salesforce	Lead	<code>LeadSource</code>	frequency + target encoding	Channel quality varies enormously
Salesforce	Lead	<code>Status</code> , <code>Rating</code>	point-in-time value via <code>LeadHistory</code>	Must be as-of ; current value leaks
Salesforce	Lead	<code>IsConverted</code> , <code>ConvertedDate</code> , <code>ConvertedOpportunityId</code>	LABEL only	Post-outcome; never a feature
Salesforce	Opportunity	<code>CreatedDate</code> , <code>StageName</code> , <code>IsWon</code> , <code>CloseDate</code> , <code>Amount</code>	label + <code>E[value]</code> target	Outcome definition & value model
Salesforce	OpportunityHistory	stage + <code>CreatedDate</code>	as-of stage reconstruction	Point-in-time correctness
Salesforce	OpportunityLineItem	<code>Product2Id</code> , <code>Quantity</code> , <code>TotalPrice</code>	product-family mix, deal size	Hardware SKU signal
Salesforce	CampaignMember	<code>HasResponded</code> , <code>FirstRespondedDate</code> , <code>CreatedDate</code>	# campaigns responded (as-of)	Engagement breadth
Salesforce	Task/Event	<code>ActivityDate</code> , <code>WhoId</code> , <code>WhatId</code>	# touches, days-to-first-touch, SLA	Sales-activity intensity

Source	Object	Field / API name	Derived feature	Rationale
HubSpot	contact	<code>hs_analytics_num_page_views</code> , <code>hs_analytics_num_visits</code>	7/30/90-day windowed counts	Web intent
HubSpot	contact	<code>hs_email_click</code> (not <code>hs_email_open</code>)	decayed click rate	Click reliable; open is not (MPP)
HubSpot	contact	<code>num_conversion_events</code> , <code>recent_conversion_event_name</code>	conversion count & type	Content/form intent
HubSpot	contact	<code>hs_analytics_source</code>	source dummies	Attribution/quality
HubSpot	contact	lifecycle date stamps	as-of lifecycle label	Point-in-time transitions
Enrichment	—	firmographics/technographics	ICP-fit + tech-install flags	Fill CRM gaps, add fit
Bombora/6sense	—	intent surge topics	account intent score	Off-property demand
Custom	<code>Install_Base__c</code>	device counts, last purchase date	refresh-cycle features	Hardware-specific propensity
Custom	Opportunity <code>__c</code>	<code>Reseller_Account__c</code> , <code>Sell_Through_Flag__c</code>	channel-deal indicators	Channel obfuscation handling

3. Feature Selection and Feature Engineering (Primary Emphasis)

3.1 Recommended feature catalogue (grouped)

- Firmographic/fit:** employee bands, revenue bands, industry/SIC/NAICS, region, company hierarchy (parent vs. subsidiary), public/private.
- Technographic:** installed UC platform, existing peripheral vendors, cloud stack, device management (MDM) presence.
- Demographic/persona:** normalized job title, seniority (IC/manager/director/VP/C-level), function (IT/Facilities/Procurement/Line-of-business), department.
- Behavioral/engagement:** page views, sessions, content downloads, webinar/event attendance, email clicks, chat, demo/quote requests — all windowed and decayed.

5. **Intent:** first-party (pricing-page views, demo requests) and third-party (Bombora surge, G2 category views).
6. **RFM-style engagement aggregates:** recency of last meaningful action, frequency of actions, "monetary" proxy (quote value requested, product interest).
7. **Velocity/trend:** deltas, slopes, and acceleration of engagement across consecutive windows (e.g., page-views last-7d vs. prior-7d ratio) — often the strongest intent signals.
8. **Channel/source attribution:** original + latest source, campaign, partner-sourced flag.
9. **Sales-activity:** calls/meetings/emails logged, response times, SLA-to-first-touch, # touches before outcome.
10. **Account-level rollups:** count of engaged contacts, distinct personas engaged, seniority of most-engaged contact, buying-group coverage.
11. **Buying-group/committee:** economic-buyer engaged (Y/N), IT+LoB both engaged, multi-thread depth.
12. **Seasonality/temporal:** fiscal quarter, education budget season, month-of-year, days since lead creation.
13. **Prior-relationship / install base:** existing customer flag, device count, time-since-last-purchase, refresh-due indicator, warranty status.

3.2 Time-windowed aggregations and decay

Compute engagement aggregates over **7/14/30/60/90-day windows ending at the observation date** (t_0) (never "to today"). Two complementary encodings:

- **Windowed counts/sums:** `page_views_30d`, `email_clicks_14d`, etc.
- **Exponentially decay-weighted score:** $\sum_i w_i$ where $w_i = \exp(-\lambda \cdot \text{age_days}_i)$ and $\text{age} = t_0 - \text{event_date}$. Choose the half-life ($(\ln 2 / \lambda)$) to match the motion (e.g., 14–30 days for in-market intent). Decay captures "recent engagement matters more" smoothly, avoiding hard window cliffs. **Critical:** every aggregate is computed as of (t_0), so an event at ($t_0 + 3$) days must be invisible — enforced by `ASOF JOIN`/windowed SQL (Section 6), not by `WHERE date <= CURRENT_DATE`.

3.3 Feature selection methodology (in rigorous order)

1. **Leakage audit FIRST** (Section 3.4). No statistical selection is valid until leaky features are removed; a leaky feature will dominate every importance ranking and validate beautifully offline while failing in production.
2. **Univariate screening:** Information Value (IV)/Weight of Evidence (WOE), mutual information, chi-square for categoricals. Cheap first cut; drop near-zero-signal features. (Caveat: univariate methods miss interactions, so use as a floor filter, not a final selector.)
3. **Correlation / multicollinearity:** correlation clustering + Variance Inflation Factor (VIF); within a cluster of collinear features keep the most interpretable/stable. Matters more for the logistic baseline than for trees, but reduces SHAP attribution instability.
4. **Model-based selection:** L1-regularized logistic regression (sparsity), tree-based importance, permutation importance, and SHAP-based selection.

5. **Stability selection / recursive feature elimination (RFE):** repeat selection across bootstraps/time-folds; keep features selected consistently. Guards against features that look good in one period only.
6. **Adjusted-predictiveness checks:** re-verify each retained feature adds out-of-time lift, not just in-fold.

Why permutation importance and SHAP beat raw gain/split importance. GBDT built-in importances (gain, split count, cover) are **biased toward high-cardinality and continuous features** (more split opportunities) and are computed on *training* data, so they reward overfitting. **Permutation importance** measures the drop in a chosen metric on *held-out* data when a feature is shuffled — it reflects genuine predictive contribution and can be computed on the metric you care about (AUC-PR). **SHAP** provides consistent, locally-accurate, additive attributions with correct sign and magnitude, enabling both global selection and the per-lead "top-3 reasons" explanations sales needs (Section 8). Prefer permutation + SHAP; treat gain importance as a rough diagnostic only.

3.4 Data leakage — the single biggest failure mode (treat with paranoia)

Leakage = using information at training time that would not have been available at scoring time for that lead. Sources specific to this stack:

- **Post-conversion fields:** Salesforce (`IsConverted`), (`ConvertedDate`), (`ConvertedOpportunityId`), (`ConvertedAccountId`) are set *at the moment of the outcome* — they are the label, never features.
- **Fields updated by sales after qualification:** (`Lead.Status`) moving to "Qualified," (`Rating`) upgraded, opportunity created — these change *because of* the outcome.
- **Backfilled-on-conversion fields:** several Lead fields get copied/populated during conversion.
- **"Last activity date" style fields** (`LastActivityDate`), HubSpot "time last seen"): if read as current value they encode future activity. Must be reconstructed as-of (`t0`).
- **Target-encoding leakage:** computing category means over the whole dataset (including the row's own label / future rows) leaks the target. Encoders must be fit **inside each CV fold on past data only**.
- **HubSpot's own** (`hubspotscore`)/(`hs_predictivecontactscore`): these are updated continuously; their *current* value reflects post-`t0` behavior. Use only their as-of value or exclude.

Why you cannot just query current CRM state. The CRM shows *today's* values. A lead that converted last month now has (`Status='Qualified'`), (`Rating='Hot'`), a linked opportunity, and inflated activity counts — all *consequences* of conversion. Training on that teaches the model to "predict" the past. You must reconstruct **what each field looked like at (`t0`)**, which requires event-timestamped history.

Building a point-in-time / as-of feature store in Snowflake:

- **Event-timestamped tables:** ingest every activity/email/pageview/stage-change with its true event timestamp; never overwrite.

- **ASOF JOIN** (GA since 13 May 2024): joins each lead's `(t0)` to the *last known value at or before* `(t0)` using `(MATCH_CONDITION (left.t0 >= right.event_ts))` — the native, optimized way to get as-of feature values. Snowflake's GA benchmark used 39 million stock quotes and 2.2 million trades and reported **ASOF JOIN queries running 12× faster than a LEFT-JOIN workaround (16× faster when the data was scaled tenfold)**.
- **Snapshot tables:** daily snapshots of slowly-changing dimensions (SCD-2 with `(valid_from/valid_to)`).
- **Change history:** Salesforce `(OpportunityHistory/LeadHistory)`, HubSpot lifecycle date stamps, and CDC feeds reconstruct historical states.
- **Snowflake Time Travel and streams:** Time Travel (retention up to 90 days on Enterprise) helps recover recent prior states, and **streams** capture CDC deltas for incremental snapshotting. *Caveat: Time Travel is bounded (max 90 days) and is a recovery mechanism, not a substitute for explicit, permanent SCD snapshot tables — build the snapshots.*

3.5 High-cardinality categoricals

Industry, job title, lead source, country, campaign can have thousands of levels.

- **In-fold target/ordinal/frequency encoding:** compute encodings only on the training partition of each temporal fold (Section 3.4). CatBoost's ordered target encoding does this natively and is a strong reason to consider it here.
- **Embeddings:** learned dense vectors for very high-cardinality IDs when volume supports it.
- **Group rare levels:** collapse levels below a frequency threshold into "Other."
- **Title normalization/seniority parsing:** map free-text titles → (seniority, function) via a controlled vocabulary/regex/LLM-assisted mapping; raw titles are too sparse and multilingual (relevant for a global hardware vendor).

3.6 Missing data (missing-not-at-random)

CRM missingness is rarely random: a blank `(AnnualRevenue)` often means "small/unenriched," a missing title often correlates with low-quality form fills. Therefore:

- **Add explicit missingness indicators** (`(is_null_revenue)`) — missingness itself is signal.
- For **trees (LightGBM/XGBoost)**, prefer leaving NaNs in place; they learn optimal default directions for missing values — usually better than naive imputation.
- For the **logistic baseline**, impute (median + indicator) and document.
- Never impute the *label*.

3.7 Class imbalance

Conversion is rare (single-digit %); positives are precious.

- **Why AUC-PR beats ROC-AUC:** ROC-AUC can look excellent (0.9+) while the model is useless at the top of the list, because the huge true-negative pool inflates it. **Precision-Recall AUC / average precision** focuses on the positive class and is far more sensitive to performance where sales actually operates (the top deciles). Report both; optimize/threshold on AUC-PR and lift@k.

- `scale_pos_weight` vs. SMOTE vs. undersampling: prefer cost-sensitive learning (`scale_pos_weight` / class weights) in the GBDT — it preserves the real data distribution and calibration best. SMOTE (synthetic minority oversampling) often hurts on high-dimensional sparse CRM data and, like any resampling, **distorts the base rate and breaks probability calibration** — the model's outputs no longer mean "probability." Undersampling the majority is acceptable for speed but requires a **calibration correction** (a prior-correction/offset or a post-hoc calibrator fit on the true distribution).
 - **Why resampling breaks calibration:** changing the positive:negative ratio changes the implied prior; the model's raw scores then systematically over-estimate $P(\text{convert})$. Since scores go to sales as probabilities and feed expected-value math, you must either avoid resampling or recalibrate on a natural-distribution set (Section 4.10).
-

4. Model Selection (Primary Emphasis)

4.1 Primary recommendation: gradient-boosted decision trees

Recommendation: a GBDT (LightGBM default; XGBoost or CatBoost as alternatives) is the workhorse. Evidence: Grinsztajn, Oyallon & Varoquaux, "Why do tree-based models still outperform deep learning on typical tabular data?" (NeurIPS 2022 Datasets & Benchmarks) benchmarked tree models vs. modern deep nets across **45 datasets** and found **tree-based models remain state-of-the-art on medium-sized (~10K-sample) tabular data even before accounting for their much lower compute**, attributing this to three inductive biases: (1) robustness to uninformative features, (2) preserving the orientation of the data (trees are not rotation-invariant, unlike MLPs, which helps on tabular data), and (3) ease of learning irregular (non-smooth) functions. Shwartz-Ziv & Armon, "Tabular Data: Deep Learning Is Not All You Need" (Information Fusion, vol. 81, pp. 84–90, 2022) independently found **XGBoost outperformed several purpose-built deep tabular models across datasets — including the datasets from the papers proposing those deep models — while requiring much less tuning** (they note an ensemble of deep models + XGBoost did best). CRM lead data is exactly this regime: heterogeneous, tabular, medium-sized, many weak/uninformative features, non-smooth relationships.

LightGBM vs. XGBoost vs. CatBoost:

Model	Categorical handling	Speed	Overfitting behavior	Use when
LightGBM	Native categorical (integer-encoded), leaf-wise growth	Fastest on large/wide data (histogram + leaf-wise)	Leaf-wise can overfit small data; control <code>(num_leaves)</code> , <code>(min_child_samples)</code>	Default; large feature sets, need speed
XGBoost	No native categorical historically (needs encoding); newer versions add support	Fast, very mature	Level-wise, robust with regularization (<code>(lambda)</code> , <code>(alpha)</code>)	Maximum stability/tooling, well-understood
CatBoost	Best-in-class: ordered target encoding, handles high-cardinality natively, reduces target leakage	Slower to train, fast inference	Ordered boosting resists overfit on small/noisy data	Many high-cardinality categoricals (industry, title, campaign), smaller positive counts

For this problem — many high-cardinality categoricals and possible low positive counts in enterprise segments — **CatBoost is a serious contender**; benchmark it against LightGBM.

4.2 Regularized logistic regression baseline (mandatory)

Always build an **L1/L2 logistic regression** (with WOE-encoded features) first. Reasons: (a) it sets a defensible performance floor — if the GBDT can't beat a well-built logistic model, the added complexity isn't justified; (b) it's transparent and easy to get sign-off on; (c) coefficients sanity-check feature directions and expose leakage (a feature with an implausibly huge coefficient is often leaky); (d) it's a fallback in cold-start/low-data segments (Section 8). A baseline is not optional — it is your control.

4.3 Survival / time-to-event framing

For long, right-censored B2B cycles, a fixed-window binary classifier throws away information: a lead created 20 days ago that hasn't converted is not a "negative" — it's **censored** (outcome unknown). Survival models handle this correctly.

- **Cox proportional hazards:** models the hazard (instantaneous conversion rate); gives relative risk. Assumes proportional hazards and log-linear covariates.
- **Discrete-time survival:** reframe as a person-period binary classification (one row per lead per time bucket, predicting "convert this bucket | survived so far") — you can run this with your existing GBDT, which is why it's the pragmatic choice; it naturally yields time-varying covariates and handles censoring. It converges to Cox with fine time bins.
- **Kaplan-Meier:** non-parametric survival curves for cohort diagnostics (e.g., "channel-sourced vs. direct leads' conversion-by-day curves").

- **Random survival forests / GBDT survival:** nonparametric, capture interactions.

When materially better than fixed-window binary: when (a) cycles are long and highly variable (enterprise/education/tender), (b) you must score *recent* leads whose window isn't complete, and (c) you care about *when*, not just *whether* (e.g., "likely to convert in Q3 vs. Q1"). Recommendation: **fixed-window GBDT for the high-volume SMB/mid-market motion; discrete-time survival for the long-cycle enterprise/channel motion.**

4.4 Uplift / causal modeling — which leads to CONTACT

Propensity answers "who will convert?" It does **not** answer "who will convert *because* we contacted them?" Some high-propensity leads are **sure things** (would convert with no outreach — outreach is wasted); the value is in **persuadables**. Uplift/incrementality models estimate the *treatment effect* of an action.

- **Approaches:** two-model (T-learner: separate treated/control models, subtract), class transformation (revert-label/S-learner variants), uplift trees/forests, causal forests (X-learner).
- **Evaluation:** Qini curve / Qini coefficient and Area Under the Uplift Curve (AUUC) — rank leads by predicted uplift and measure cumulative *incremental* conversions vs. random targeting. A Qini coefficient of 0 equals random; negative means the model is worse than random.
- **Why propensity ≠ incrementality:** a lead already at 95% will-convert has near-zero uplift; a 40% lead that jumps to 60% with a call has high uplift. Ranking by propensity over-invests in sure things and do-not-disturbs.
- **When to invest in uplift:** only after you have (a) a working propensity model, (b) **experimental variation** (a real randomized holdout of contacted vs. not-contacted — you cannot learn uplift cleanly from observational data), and (c) a decision that is genuinely about *allocating scarce outreach*. Uplift is a phase-2/3 sophistication, not a v1.

4.5 Two-stage / expected-value scoring

$\text{ExpectedValue} = P(\text{convert within } W) \times E[\text{deal value} \mid \text{convert}]$. Train the value model (regression on `Opportunity.Amount`/`OpportunityLineItem.TotalPrice`, log-transformed) separately, then multiply. This prioritizes by **expected revenue**, not raw probability — critical for hardware where a 30-seat VC-room deal dwarfs a single-webcam SMB deal. This is the natural bridge to the CLV model (Section 1.5): swap $E[\text{deal value}]$ for $E[\text{CLV}]$ to prioritize long-term value.

4.6 Multi-task / hierarchical, and when rules are the right answer

- **Multi-task / hierarchical:** jointly model MQL→SQL and SQL→Opp, or account- and lead-level, sharing representations. Worth it at scale with rich data; adds complexity.
- **When a simple points-based/rules model is genuinely correct: cold start** (no history), **very low positive counts**, or when **sales trust** must be earned first. A transparent rules/points model (fit points + engagement points, thresholded to A/B/C/D) is legitimately the right v0. It's also a useful *feature* and a *challenger* for the ML model.

4.7 Deep learning, TabPFN, tabular foundation models (honest assessment)

- **When DL is worth it:** genuinely large **behavioral sequence** data — model raw event sequences (page/email/product-usage clickstreams) with sequence models/transformers, which can beat GBDTs when the *ordering and timing* of events carries signal that hand-engineered aggregates lose. This is the strongest DL case for lead scoring, and even then a GBDT on well-engineered sequence features is a tough baseline.
- **TabPFN / tabular foundation models:** TabPFN (Hollmann et al., ICLR 2023) is a transformer pre-trained to do in-context Bayesian inference on **small** tabular datasets in one forward pass; a 2025 *Nature* paper extended it as a tabular foundation model. Genuinely useful for **small/cold-start** segments, but originally constrained on dataset size/feature count and still maturing for production. Treat as an experiment/challenger, not the primary system.
- **Verdict:** default to GBDTs; reserve DL for large sequence data; pilot TabPFN for small segments. The evidence (Section 4.1) does not support deep learning as the default for this data.

4.8 Snowflake-native vs. external training

- **Snowflake Cortex ML** (`CLASSIFICATION`): a no-code, SQL-callable ML function — **it is powered by a gradient boosting machine**, trained for binary classification with an **area-under-the-curve loss function** (multi-class uses logistic loss), and returns per-class probabilities plus feature importances (values 0-1, summing to 1). Excellent **fast baseline** with zero infra. Limits: less control (no custom loss, limited tuning, no point-in-time CV orchestration), and models from ML Functions (like `FORECAST`) don't appear in the Model Registry. (Snowflake)
- **Snowpark ML** / (`snowflake-ml-python`): run **XGBoost/LightGBM/CatBoost inside Snowpark** (Container Runtime for heavier workloads), use the **Snowflake Feature Store** (entities + feature views) and **Model Registry** (versioning, metrics, SQL/Python/REST inference, and **SHAP-based** (`explain()`) **available by default for models deployable to the warehouse**, computing Shapley values). This is the recommended production path.
- **vs. SageMaker/Vertex/Databricks:** external platforms offer richer experiment tracking, custom training images, and mature MLOps, but require **moving governed PII data out of Snowflake** (residency/consent risk, Section 7) and add egress + a second system to secure. **Keeping everything in Snowflake** minimizes data movement, keeps one governance/RBAC perimeter, and simplifies point-in-time joins (features already live there). Trade-off: Snowflake ML tooling is younger than SageMaker/Vertex/MLflow, and very custom deep-learning or distributed training may still be easier externally. **Recommendation: train and score in Snowflake; use external MLflow/W&B only for experiment tracking if the team already relies on it.**

4.9 Hyperparameter tuning and temporal cross-validation

- **Tuning: Optuna** (Bayesian/TPE) or similar; tune (`num_leaves`), (`max_depth`), (`learning_rate`), (`min_child_samples`), `subsample`/`colsample`, regularization; use early stopping on a temporal validation fold.
- **CRITICAL — temporal CV, not random k-fold.** Split by time: train on (`[..t_train]`), validate

on `[[t_train..t_val]]`, always past→future (expanding or rolling window). **Why random k-fold inflates performance:** it lets the model see future rows and future-informed encodings during training, and it ignores concept drift; the reported metric is optimistic and will not reproduce in production. Random splits also interact catastrophically with target encoding and any residual leakage. Temporal CV mirrors deployment reality (you always predict the future from the past) and is the honest estimate.

4.10 Probability calibration

Sales receives scores as **probabilities** and expected-value math multiplies them, so calibration matters as much as ranking.

- **Platt scaling** (fit a logistic on the model's scores): parametric, data-efficient, good when miscalibration is roughly sigmoidal; safest with limited positives.
- **Isotonic regression:** non-parametric, more flexible, can fix arbitrary monotonic miscalibration, but needs more data and can overfit small sets.
- **Diagnostics: reliability diagrams** (predicted vs. observed frequency per bin), **Brier score** (lower = better), log loss. Recompute calibration **after any resampling** (Section 3.7) on a natural-distribution set. Fit the calibrator on a **held-out, time-appropriate** slice, never on training data.

4.11 Model comparison summary

Approach	Primary use	Strengths	Weaknesses	Verdict
Logistic (L1/L2)	Baseline / cold-start fallback	Transparent, fast, leak-detector	Misses interactions	Mandatory control
LightGBM/XGBoost/CatBoost	Primary propensity	SOTA on tabular, handles NaNs & categoricals	Needs calibration, careful CV	Recommended core
Discrete-time survival	Long-cycle enterprise/channel	Handles censoring & timing	More setup	Use for long cycles
Uplift / causal	Outreach allocation (phase 2+)	Targets incrementality	Needs experiments	After v1 + RCT data
Expected-value (P×\$)	Revenue prioritization	Aligns to \$	Compounds errors	Ranking layer
Cortex CLASSIFICATION	Fast SQL baseline	Zero infra, GBM-powered	Limited control	Baseline / prototyping
Deep sequence / TabPFN	Big sequences / small segments	Sequence signal / cold-start	Immature/infra-heavy	Experiment only

5. Evaluation

5.1 Offline metrics

- **AUC-PR / average precision** (primary on imbalanced data), **ROC-AUC** (secondary/ comparability).
- **Lift@k and cumulative gain curves** — the **most sales-legible** metric: "the top 10% of scored leads contain 5× the conversions of random." Speaks directly to prioritization.
- **Precision/recall at the operating threshold**, **KS statistic** (separation), **Brier score / log loss** (calibration), **reliability/calibration curves**.
- **Qini / uplift curves** where uplift modeling applies (Section 4.4).

5.2 Choosing the operating threshold from sales capacity

Do **not** use 0.5. Set the threshold from **capacity**: if SDRs can work N leads/day and you score M/day, the threshold is the score cutoff that passes ~N per day (i.e., the top-k that fits capacity), read off the cumulative-gain curve. This turns the model into a capacity-aware prioritizer and is the single most impactful "productionization" decision.

5.3 Score banding

Convert calibrated probability to **tiers (A/B/C/D or Hot/Warm/Cool/Cold)** for sales legibility. Set cutoffs by **decile analysis** on out-of-time data: choose bands where conversion rates are meaningfully separated and volumes match capacity (e.g., A = top decile at ~40% conversion, worked same-day; D = bottom half, nurture only). Recalibrate bands as base rates shift.

5.4 Business metrics (the real scorecard)

MQL→SQL rate, SQL→opportunity rate, win rate, pipeline generated, sales-cycle length, **cost per SQL**, rep time saved, and — most importantly — **incremental pipeline vs. the pre-model baseline / holdout**. Offline AUC-PR is a means; these are the ends.

5.5 Online validation and the feedback-loop trap

- **Randomized holdout / A/B test the routing**: run score-driven routing for most leads but keep a **permanent random control group** and champion/challenger models.
- **The self-fulfilling prophecy problem (explained in depth)**: once you route by score, **low-scored leads never get worked, so they never convert — regardless of their true potential**. The observed data then "confirms" the model (low scores → 0 conversions), the next model trains on this biased data and gets *more* confident that low scores are hopeless, and genuinely good leads that happened to score low are permanently starved. This is **selection bias / a closed feedback loop**, and it degrades the model over time while looking like success.
- **Mitigations**: (1) a permanent **random exploration budget** — always work a small random % of *all* leads including low-scored ones, to keep collecting unbiased labels; (2) **propensity-weighted / inverse-propensity evaluation** — weight outcomes by the probability a lead was worked, to de-bias offline estimates; (3) monitor conversion in the low-score band from the exploration set as an early-warning of model rot.

5.6 Fairness / compliance

Do not use protected attributes (or close proxies) — for individuals this can be legally hazardous (Section 7). B2B fit features (company size, industry) are generally fine, but persona/geography features can proxy protected classes; audit for it, document, and keep scoring *organization*-centric where possible.

6. Snowflake Implementation — Sample Code

6.1 Layered architecture and naming

```
RAW          -- landing, 1:1 with source, append-only, event-timestamped (Fivetran /
HubSpot data share / SFDC connector)
STAGING      -- typed, deduped, renamed, light cleaning (dbt staging models; dynamic
tables)
CORE         -- conformed dimensional model: dim_account, dim_lead, dim_contact,
fct_activity, fct_opportunity, fct_stage_history
FEATURES     -- point-in-time feature views (Feature Store), one row per entity per
as-of date
ML           -- training datasets, model artifacts (Model Registry), predictions
SERVING      -- scored tables ready for reverse ETL to SFDC/HubSpot
```

Naming: `stg_<source>__<object>`, `dim_/fct_`, `feat_<entity>__<group>`,
`ml_train_<label>_<window>`, `serving_lead_scores`.

6.2 Point-in-time label construction (opportunity-creation within W days)

```
sql
```

```
-- Observation grid: for each lead, choose observation date(s) t0.
-- v1: t0 = lead creation date. (Later: multiple t0 per lead for survival/person-period)
-- Label = 1 if an opportunity was created for that lead's account within [t0, t0 + W)
-- Leads whose window is not yet complete (t0 > today - W) are EXCLUDED (censored / immature window)
```

```
CREATE OR REPLACE DYNAMIC TABLE ml.ml_labels_opp_60d
    TARGET_LAG = '24 hours'
    WAREHOUSE = wh_ml
    REFRESH_MODE = INCREMENTAL
AS
WITH params AS (SELECT 60 AS window_days),
lead_obs AS (
    SELECT l.lead_id,
           l.account_id,
           l.created_date::DATE AS t0
    FROM   core.dim_lead l
),
opp_created AS ( -- earliest opportunity creation per account (point-in-time safe:
    SELECT account_id, MIN(created_date::DATE) AS first_opp_date
    FROM   core.fct_opportunity
    GROUP BY account_id
)
SELECT o.lead_id,
       o.account_id,
       o.t0,
       CASE WHEN oc.first_opp_date IS NOT NULL
            AND oc.first_opp_date > o.t0
            AND oc.first_opp_date <= DATEADD(day, p.window_days, o.t0)
            THEN 1 ELSE 0 END AS label_opp_60d
FROM   lead_obs o
CROSS JOIN params p
LEFT JOIN opp_created oc ON oc.account_id = o.account_id
WHERE  o.t0 <= DATEADD(day, -p.window_days, CURRENT_DATE); -- exclude immature window
```

6.3 Feature engineering patterns

Time-windowed + decay-weighted engagement, point-in-time.

```
sql
```

```
-- HubSpot page-view events already flattened to (lead_id, event_ts) in core.fct_web_
-- For each lead observation (lead_id, t0) compute 7/30/90-day counts and a 14-day-ha
-- counting ONLY events strictly before t0 (no leakage).
```

```
CREATE OR REPLACE DYNAMIC TABLE features.feat_lead__web_engagement
  TARGET_LAG = '24 hours' WAREHOUSE = wh_ml REFRESH_MODE = INCREMENTAL
AS
SELECT o.lead_id, o.t0,
       COUNT_IF(e.event_ts >= DATEADD(day,-7,o.t0)) AS
       COUNT_IF(e.event_ts >= DATEADD(day,-30,o.t0)) AS
       COUNT_IF(e.event_ts >= DATEADD(day,-90,o.t0)) AS
       -- exponential decay: weight = 0.5 ^ (age_days / 14)
       SUM( POWER(0.5, DATEDIFF(day, e.event_ts, o.t0) / 14.0) ) AS
       -- velocity: last-7d vs prior-7d ratio (acceleration of intent)
       DIV0( COUNT_IF(e.event_ts >= DATEADD(day,-7,o.t0)),
            NULLIF(COUNT_IF(e.event_ts >= DATEADD(day,-14,o.t0)
                          AND e.event_ts < DATEADD(day,-7,o.t0)),0) ) AS
FROM   ml.ml_labels_opp_60d o
LEFT   JOIN core.fct_web_events e
      ON e.lead_id = o.lead_id
      AND e.event_ts < o.t0 -- STRICT point-in-time cutoff
GROUP BY o.lead_id, o.t0;
```

As-of slowly-changing attribute (Lead.Status reconstructed from LeadHistory) via `ASOF JOIN`.

```
sql

-- core.fct_lead_status_history: (lead_id, status_value, valid_from_ts) built from Sa
SELECT o.lead_id, o.t0, h.status_value AS status_as_of_t0
FROM   ml.ml_labels_opp_60d o
ASOF JOIN core.fct_lead_status_history h
      MATCH_CONDITION (o.t0 >= h.valid_from_ts) -- last known status at or before t0
      ON o.lead_id = h.lead_id;
```

Semi-structured HubSpot JSON with `LATERAL FLATTEN` / `VARIANT`.

```
sql
```

```
-- RAW.hubspot_contacts.payload is VARIANT (nested properties + associations array).
CREATE OR REPLACE DYNAMIC TABLE staging.stg_hubspot__contacts
  TARGET_LAG='1 hour' WAREHOUSE=wh_transform REFRESH_MODE=INCREMENTAL
AS
SELECT payload:vid::NUMBER AS hs_contact_id,
  payload:properties:lifecyclestage:value::STRING AS lifecyclestage,
  payload:properties:hs_analytics_num_page_views:value::INT AS num_page_views,
  payload:properties:hs_email_click:value::INT AS email_clicks,
  payload:properties:num_conversion_events:value::INT AS num_conversion_ev
  payload:properties:recent_conversion_event_name:value::STRING AS recent_conver
  assoc.value:company_id::NUMBER AS associated_compan
FROM raw.hubspot_contacts,
  LATERAL FLATTEN(input => payload:associations:companies, OUTER => TRUE) assoc;
```

Account-level buying-committee rollup (using QUALIFY where a per-partition filter is needed).

```
sql

SELECT o.account_id, o.t0,
  COUNT(DISTINCT c.contact_id) AS engage
  COUNT(DISTINCT c.seniority) AS distin
  MAX(IFF(c.seniority='C-Level',1,0)) AS has_ex
  MAX(w.pv_decay_hl14) AS max_co
FROM ml.ml_labels_opp_60d o
JOIN core.dim_contact c ON c.account_id = o.account_id
LEFT JOIN features.feats_lead__web_engagement w ON w.lead_id = c.lead_id AND w.t0 =
GROUP BY o.account_id, o.t0;
```

6.4 Training-set assembly (one row per lead per as-of date)

```
sql

CREATE OR REPLACE TABLE ml.ml_train_opp_60d AS
SELECT lab.lead_id, lab.account_id, lab.t0, lab.label_opp_60d,
  web.pv_7d, web.pv_30d, web.pv_90d, web.pv_decay_hl14, web.pv_velocity_7v7,
  fit.employees_band, fit.industry, fit.tech_uc_platform,
  stat.status_as_of_t0,
  grp.engaged_contacts, grp.has_exec_engaged, grp.max_contact_intent
FROM ml.ml_labels_opp_60d lab
LEFT JOIN features.feats_lead__web_engagement web USING (lead_id, t0)
LEFT JOIN features.feats_account__firmographic fit USING (account_id, t0)
LEFT JOIN features.feats_lead__status_asof stat USING (lead_id, t0)
LEFT JOIN features.feats_account__buying_group grp USING (account_id, t0);
```

6.5 Snowpark Python: train LightGBM, register, batch-score

```
python

from snowflake.snowpark import Session
from snowflake.ml.registry import Registry
import lightgbm as lgb, numpy as np

session = Session.builder.getOrCreate()

# Pull the point-in-time training set; ORDER BY t0 for temporal split (never random).
df = session.table("ML.ML_TRAIN_OPP_60D").to_pandas().sort_values("T0")
cut = df["T0"].quantile(0.8) # time-based split
train, valid = df[df.T0 <= cut], df[df.T0 > cut]

features = ["PV_7D", "PV_30D", "PV_90D", "PV_DECAY_HL14", "PV_VELOCITY_7V7",
            "EMPLOYEES_BAND", "INDUSTRY", "TECH_UC_PLATFORM", "STATUS_AS_OF_T0",
            "ENGAGED_CONTACTS", "HAS_EXEC_ENGAGED", "MAX_CONTACT_INTENT"]
cats = ["INDUSTRY", "TECH_UC_PLATFORM", "STATUS_AS_OF_T0"]
for c in cats: # LightGBM native categoricals
    train[c] = train[c].astype("category"); valid[c] = valid[c].astype("category")

pos = train.LABEL_OPP_60D.sum(); neg = len(train)-pos
model = lgb.LGBMClassifier(
    n_estimators=2000, learning_rate=0.02, num_leaves=31, min_child_samples=100,
    subsample=0.8, colsample_bytree=0.8, reg_lambda=5.0,
    scale_pos_weight=neg/pos, # cost-sensitive, NOT SMOTE (keep
    objective="binary")
model.fit(train[features], train.LABEL_OPP_60D,
          eval_set=[(valid[features], valid.LABEL_OPP_60D)],
          eval_metric="average_precision", # AUC-PR, not accuracy
          callbacks=[lgb.early_stopping(100)])

# Calibrate on the held-out (natural-distribution) validation slice.
from sklearn.isotonic import IsotonicRegression
raw = model.predict_proba(valid[features])[:,1]
cal = IsotonicRegression(out_of_bounds="clip").fit(raw, valid.LABEL_OPP_60D)

# Register in the Snowflake Model Registry (versioned; supports SHAP explain()).
reg = Registry(session=session, database_name="ML", schema_name="REGISTRY")
mv = reg.log_model(model, model_name="LEAD_PROPENSITY_OPP60",
                  version_name="v1", sample_input_data=train[features].head(100),
                  metrics={"val_ap": float(np.mean(raw))}, conda_dependencies=["ligh
```

Batch scoring can then be invoked from SQL against the registered model (`(mv.run(...))` / a generated inference function), writing to `SERVING.serving_lead_scores`, or scheduled via a Snowflake Task.

6.6 Cortex ML **CLASSIFICATION** fast baseline

```
sql

-- Zero-infra GBM baseline (AUC loss under the hood). Use ONLY the point-in-time train data
CREATE OR REPLACE SNOWFLAKE.ML.CLASSIFICATION lead_baseline(
  INPUT_DATA => TABLE(ML.ML_TRAIN_OPP_60D),
  TARGET_COLNAME => 'LABEL_OPP_60D');

CREATE OR REPLACE TABLE serving.serving_lead_scores_baseline AS
SELECT lead_id,
  lead_baseline!PREDICT(OBJECT_CONSTRUCT(*)):class::INT
  lead_baseline!PREDICT(OBJECT_CONSTRUCT(*)):probability:"1"::FLOAT
FROM features.feat_scoring_today;      -- today's leads, features computed as-of to
```

6.7 Write-back to Salesforce and HubSpot

- **Reverse ETL: Hightouch or Census** read `(SERVING.serving_lead_scores)` and upsert to CRM fields on a schedule; both natively support Salesforce and HubSpot and dbt-model-based syncs. Hightouch publishes 200+ (recent reviews cite up to 350+) destination connectors. **Note: Census was acquired by Fivetran (agreement announced May 1, 2025) and rebranded "Fivetran Activations" with Monthly-Active-Rows pricing** — factor that into vendor selection. **HubSpot's native Snowflake data share is inbound-only** (HubSpot→Snowflake, read-only, requires Operations Hub Enterprise or Data Hub Enterprise); pushing scores *back* to HubSpot requires HubSpot's separate data-ingestion connector (which needs network/firewall access to Snowflake) or a reverse-ETL tool — the share cannot be reversed. **Salesforce Data Cloud** is an alternative activation path for SFDC.
- **Custom fields to create:**
 - Salesforce `(Lead)/(Account): Predictive_Score__c` (Number, calibrated prob), `(Predictive_Tier__c)` (Picklist A/B/C/D), `(Score_Reason_1__c/2__c/3__c)` (Text, SHAP top-3), `(Score_Model_Version__c)`, `(Score_Updated_At__c)`, `(Expected_Value__c)`.
 - HubSpot custom contact/company properties: `(predictive_score)`, `(predictive_tier)`, `(score_reason_1/2/3)`, `(score_model_version)`, `(score_updated_at)`. Keep these clearly namespaced and *distinct* from HubSpot's built-in `(hubspot_score)/` `(hs_predictive_contactscore)` to avoid confusion.
- **Write the reasons alongside the score** — an unexplained number gets ignored (Section 8).

6.8 Cost/performance

- **Clustering keys:** cluster large event/fact tables on `(entity_id, event_ts)` so point-in-time window scans prune micro-partitions instead of full scans.
- **Dynamic tables** for declarative incremental feature refresh (target lag); **streams+tasks** where you need MERGE/procedural logic (dynamic tables don't support MERGE, sequences, or `(UUID_STRING)` in incremental mode).
- **Warehouse sizing:** separate `(wh_transform)` (feature builds) from `(wh_ml)` (training) so a big

training job doesn't starve pipelines; size up only for training bursts, auto-suspend aggressively.

- **Avoid full rescans:** always bound feature queries by `event_ts < t0` and by window lower bounds; materialize expensive rollups once and reuse.

7. MLOps Pipeline

7.1 End-to-end architecture (text diagram)

```
[Salesforce] --Fivetran/SFDC connector-->|
[HubSpot]    --native Snowflake data share (inbound)-->|
[Enrichment/Intent: ZoomInfo, Bombora, 6sense] --API/Fivetran-->|
                                                    v
                                                    RAW (append-only, event-
timestamped)
                                                    v (dbt + dynamic
tables)
                                                    STAGING -> CORE (dimensional)
                                                    v
                                                    FEATURES (Feature Store, point-in-
time views)
                                                    v
                                                    TRAINING (Snowpark ML, temporal CV, Optuna)
                                                    v
                                                    MODEL REGISTRY (versions, metrics, SHAP)
                                                    v
                                                    BATCH SCORING (Task/daily) + near-real-time
triggers
                                                    v
                                                    SERVING.serving_lead_scores
                                                    v (Hightouch/
Census reverse ETL)
                                                    [Salesforce fields] + [HubSpot
properties]
                                                    v
                                                    FEEDBACK CAPTURE (worked?/outcome + exploration
holdout) -> RAW
```

7.2 Orchestration and cadence

- **Orchestration:** **dbt** (on Snowflake) for transformations + tests; **Snowflake Tasks** for in-warehouse scheduling; **Airflow/Dagster** or **dbt Cloud** if you need cross-system DAGs and richer scheduling/observability. Dagster or Airflow is recommended as the top-level orchestrator once the pipeline spans ingestion + training + reverse ETL.
- **Cadence:** **daily batch scoring** for the standard propensity score; **near-real-time / event-triggered** scoring for high-intent triggers (demo request, pricing-page visit, RFP) via

streams+tasks or a triggered sync, so not leads route within minutes.

7.3 Versioning, tracking, lineage

- **Model versioning:** Snowflake Model Registry (immutable versions, metrics, lifecycle stages dev/stage/prod).
- **Experiment tracking:** MLflow or Weights & Biases for the training-side sweeps if the team wants richer comparison than the registry provides.
- **Lineage/reproducibility:** dbt lineage + Snowflake ML Lineage + Feature Store feature views + immutable dataset snapshots, so every prediction is traceable to feature definitions, code, and training data version.

7.4 Monitoring

- **Data-quality tests:** dbt tests (`(not_null)`, `(unique)`, accepted values, referential) and/or **Great Expectations** at RAW/STAGING; fail the pipeline on violations.
- **Feature drift: Population Stability Index (PSI)** and KS per feature vs. training baseline. **Standard PSI thresholds: below 0.1 = stable, 0.1-0.25 = moderate shift (watch), above 0.25 = significant shift (investigate/retrain).** Track both vs. a fixed training baseline (long-term / regulatory) and vs. a rolling 30-day baseline (operational anomaly detection).
- **Concept drift & prediction drift:** monitor score-distribution stability, band-mix over time, and prediction-vs-actual as labels mature (delayed-label performance decay tracking).
- **Performance decay with delayed labels:** because outcomes lag by `(w)`, maintain a rolling backfill that re-computes AUC-PR/lift as labels arrive, and alert when out-of-time lift falls below a threshold.

7.5 Retraining, promotion, rollback

- **Triggers:** scheduled (e.g., monthly) + event-driven (PSI > 0.25 on key features, or lift decay beyond tolerance).
- **Champion/challenger:** train challenger, compare on the *same out-of-time* window + calibration; promote only on material, stable lift.
- **Rollback:** registry keeps prior version; a bad promotion reverts by re-pointing the serving job to the previous version.

7.6 Governance and privacy (do not skip)

- **Model documentation / model cards:** intended use, training data window, features, metrics, calibration, known limitations, fairness review.
- **RBAC in Snowflake:** least-privilege roles for RAW/PII vs. FEATURES/SERVING; column-masking policies on PII.
- **PII / GDPR / CCPA: scoring individuals is personal-data processing.** In the EU, automated scoring of people carries specific obligations (lawful basis, transparency, and rights around solely-automated decisions with significant effects). Honor **consent** (marketing/tracking consent flags must gate feature use), **data residency** (another reason to keep processing in Snowflake in-region), **right to erasure/access**, and prefer **account-level** signals over individual behavioral profiling where feasible. Document a DPIA where required. Treat

8. Rollout, Adoption, and Things to Take Care Of

8.1 Phased roadmap (realistic)

- **Phase 0 — Foundations (Weeks 1-6):** ingestion (Fivetran/HubSpot share), RAW→CORE modeling, **enable Salesforce field-history tracking now** (Lead.Status/Rating/LeadSource; you can't backfill it), lead-to-account matching, data-quality tests. Ship a transparent **rules-based** score as v0.
- **Phase 1 — First model (Weeks 6-12):** point-in-time label + feature store, logistic baseline, LightGBM/CatBoost, temporal CV, calibration; offline eval (AUC-PR, lift@k). Cortex `CLASSIFICATION` as a quick parallel baseline.
- **Phase 2 — Productionize (Weeks 12-18):** Model Registry, daily batch scoring, reverse ETL to SFDC/HubSpot with SHAP reasons, **random holdout live**, monitoring (PSI, calibration, business KPIs).
- **Phase 3 — Optimize (Months 5-9+):** expected-value layer, survival model for enterprise/channel, near-real-time triggers, then uplift modeling once you have RCT data.

8.2 Cold start / minimum data

- **Events-per-variable (EPV):** the classic rule of thumb is ≥ 10 **positive events per predictor** for stable logistic/Cox coefficients (Peduzzi et al.); this is contested — Vittinghoff & McCulloch (2007, *American Journal of Epidemiology*) showed it can be relaxed, finding severe problems mainly occur at EPV of 2-4, while other work shows low-prevalence binary predictors may need $\text{EPV} \geq 20$. GBDTs tolerate more features but still need positives.
- **Practical minimum:** a widely used practitioner floor is ≥ 100 **positive cases** (and meaningfully more total) before trusting an ML model; below that, use the **rules/points model or TabPFN** and pool segments. For a 5% conversion rate and ~30 features, that floor implies on the order of thousands of leads minimum. Segment-specific models (enterprise) will hit this later than SMB — start pooled, split as positives accumulate.

8.3 Sales adoption

- **Explainability for reps:** ship SHAP-derived "**top 3 reasons this lead scored high/low**" into the CRM next to the score. This is the difference between adoption and abandonment.
- **Transparency & feedback loops:** let reps flag wrong scores ("this is a student, not a buyer"); capture it as labeled feedback. Publish the model's wins (pipeline from top tier).
- **Change management:** co-design tiers/thresholds with sales leadership; tie routing SLAs to tiers. **An unexplained black-box score gets ignored** — reps trust what they understand and what demonstrably saves them time.

8.4 Failure-mode checklist

- ☐ **Leakage** from post-conversion/backfilled/last-activity fields (Section 3.4).
- ☐ **Random CV splits** instead of temporal — inflates offline metrics.
- ☐ **Ignoring calibration** — scores handed to sales must mean probabilities.

- ☐ **Scoring on current CRM state** instead of as-of features.
- ☐ **Duplicate/junk leads and form spam** inflating engagement and corrupting labels.
- ☐ **Bot-generated email opens — Apple Mail Privacy Protection (MPP)**: since iOS 15 (Sept 2021), Apple pre-loads email content via proxy servers, marking messages "opened" regardless of whether the user opened them, inflating open rates. Per Litmus, **over 50% of email opens now occur on an MPP-enabled device**, with Apple Mail commanding roughly 48-54% of all opens in 2025 — so **email opens are unreliable and must NOT be used as engagement/intent features**. Use **clicks, form submissions, page views, and conversions** instead (`hs_email_click`), not (`hs_email_open`).
- ☐ **Lead-to-account matching failures** corrupting account rollups.
- ☐ **CRM data hygiene** (stale/blank firmographics) — mitigate with enrichment + missingness indicators.
- ☐ **Stale enrichment** — refresh cadence and vintage tracking.
- ☐ **Seasonality** (education budget cycles, fiscal year-end) — include temporal features, evaluate out-of-time.
- ☐ **Channel/campaign shifts** — monitor source-mix drift (PSI on `LeadSource`/`hs_analytics_source`).
- ☐ **Sales-process changes invalidating labels** (new stages, redefined MQL) — version label definitions.
- ☐ **Over-scoring existing customers** — exclude/segment current customers; an install-base upsell model is a *different* model.
- ☐ **Self-fulfilling feedback loop** — permanent random exploration holdout + propensity-weighted eval (Section 5.5).

8.5 Hardware/peripherals-specific complications

- **Channel/reseller-sourced deals obscure the end customer**: the deal shows the reseller, not the end account. Capture end-customer via custom fields (`End_Customer_Account__c`), deal-registration data, and enrichment; otherwise account rollups and install-base features silently break. Consider a **channel-specific model** where end-customer signal is weak.
- **Distributor sell-through visibility**: you often see sell-*in* (to distributor) but not sell-*through* (to end user). Ingest distributor POS/sell-through feeds where available; treat missing sell-through as MNAR (missingness indicator).
- **Long enterprise procurement / tender cycles**: use survival models and longer windows for this segment; don't force it into the SMB fixed-window model.
- **Education-sector budget seasonality**: strong fiscal/tender periodicity — include fiscal-calendar and "budget-season" features and always evaluate across full-year out-of-time windows.
- **Install-base refresh cycles**: time-since-last-purchase vs. device lifecycle is one of the strongest B2B-hardware propensity signals — engineer refresh-due indicators from `Install_Base__c`/`OpportunityLineItem` history; this is also where lead scoring blends into the CLV/expansion model.

Recommendations (staged, with thresholds)

1. **Now:** enable Salesforce field-history tracking on Lead status/rating/source (remember: 20-field standard limit, ~18-month UI / 24-month API retention — so start early); stand up RAW→CORE + lead-to-account matching; ship a transparent rules score. *Threshold to advance:* clean point-in-time history flowing and ≥ 100 positives in at least one segment.
2. **Next:** build the point-in-time feature store (`ASOF JOIN`, dynamic tables), logistic baseline, then LightGBM/CatBoost with temporal CV + calibration; target the **opportunity-creation-in-60-days** label. *Advance when* the GBDT beats the baseline on out-of-time **AUC-PR** and **lift@k** and calibration (Brier) is acceptable.
3. **Then:** productionize via Model Registry + daily batch + reverse ETL with SHAP reasons; **launch the random holdout on day one of routing**. *Advance when* the scored group shows materially higher SQL/opportunity rates than the random holdout.
4. **Later:** add expected-value ranking, a survival model for enterprise/channel, near-real-time triggers, and — only after you have randomized contact experiments — uplift modeling. *Trigger retraining* when $PSI > 0.25$ on key features or out-of-time lift decays beyond tolerance.

Caveats

- Some figures are industry rules of thumb, not laws: $EPV \geq 10$ is contested (relaxable per Vittinghoff & McCulloch, who found severe problems mainly at EPV 2–4; stricter — $EPV \geq 20$ — with low-prevalence predictors), and the ≥ 100 -positives floor is a practitioner heuristic — validate empirically on your data.
- Salesforce/HubSpot **standard** field names are verified against official docs; every hardware-specific signal (install base, sell-through, channel end-customer, VC-room deployments) is **company-specific and must be built as custom objects/fields** — do not assume a standard field exists.
- PSI/AUC thresholds and calibration choices are starting points; tune to your base rates and business costs.
- Third-party vendor facts (Census acquired by Fivetran and rebranded Fivetran Activations, announced May 1, 2025; HubSpot's data share being inbound-only; Cortex `CLASSIFICATION` being GBM/AUC-loss based; `ASOF JOIN` GA May 13, 2024) are current as of mid-2026 but product capabilities change — re-verify against vendor docs at build time.
- EU/automated-decision privacy obligations are summarized, not legal advice; involve privacy/legal before scoring individuals in-region.